

EXP NO: 9	MINI PROJECT - TIME SERIES-BASED STOCK PRICE FORECASTING WITH LINEAR REGRESSION
------------------	--

AIM:

To predict future stock prices using historical time series data by implementing a **Linear Regression** model, and to evaluate its performance in forecasting trends.

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

from sklearn.metrics import (
    mean_squared_error,
    r2_score
)

# Set random seed
np.random.seed(42)

# Create historical date range
dates = pd.date_range(
    start="2020-01-01",
    periods=500,
    freq="D"
)

# Generate realistic stock price data
trend = np.linspace(100, 180, 500)
```

```
seasonal = (  
    5 * np.sin(np.linspace(0, 20 * np.pi, 500))  
)
```

```
noise = np.random.normal(  
    0,  
    2,  
    500  
)
```

```
close_price = trend + seasonal + noise
```

```
# Create stock dataset
```

```
df = pd.DataFrame({  
    "Date": dates,  
    "Close": close_price  
})
```

```
# Generate Open, High, Low and Volume
```

```
df["Open"] = df["Close"] + np.random.normal(0, 1, 500)
```

```
df["High"] = (  
    df[["Open", "Close"]].max(axis=1)  
    + np.random.uniform(0.5, 3, 500)  
)
```

```
df["Low"] = (  
    df[["Open", "Close"]].min(axis=1)  
    - np.random.uniform(0.5, 3, 500)
```

```
)
```

```
df["Volume"] = np.random.randint(  
    100000,  
    1000000,  
    500  
)
```

```
# Arrange columns
```

```
df = df[  
    [  
        "Date",  
        "Open",  
        "High",  
        "Low",  
        "Close",  
        "Volume"  
    ]  
]
```

```
# Display original dataset
```

```
print("Original Dataset:")  
print(df.head())
```

```
print("\nDataset Shape:")  
print(df.shape)
```

```
# Check missing values
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
# Convert Date to datetime
```

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
# Sort data chronologically
```

```
df = df.sort_values("Date")
```

```
# Create time-based feature
```

```
df["Days"] = (  
    df["Date"] - df["Date"].min()  
) .dt.days
```

```
# Create lag feature
```

```
df["Previous_Close"] = df["Close"].shift(1)
```

```
# Remove first row created by lag
```

```
df = df.dropna()
```

```
print("\nPreprocessed Dataset:")
```

```
print(df.head())
```

```
# Select features
```

```
X = df[  
    [  
        "Days",  
        "Open",  
        "High",  
        "Low",
```

```
        "Previous_Close",
        "Volume"
    ]
]

y = df["Close"]

# Chronological 80-20 split
split_index = int(len(df) * 0.8)

X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]

y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# Create Linear Regression model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)

print("\nLinear Regression model trained successfully.")

# Predict test data
y_pred = model.predict(X_test)
```

```
# Calculate MSE
```

```
mse = mean_squared_error(  
    y_test,  
    y_pred  
)
```

```
# Calculate RMSE
```

```
rmse = np.sqrt(mse)
```

```
# Calculate R2 score
```

```
r2 = r2_score(  
    y_test,  
    y_pred  
)
```

```
print("\n" + "=" * 50)
```

```
print("MODEL EVALUATION")
```

```
print("=" * 50)
```

```
print("Mean Squared Error (MSE):", round(mse, 4))
```

```
print("Root Mean Squared Error (RMSE):", round(rmse, 4))
```

```
print("R2 Score:", round(r2, 4))
```

```
# Create comparison dataframe
```

```
comparison = pd.DataFrame({  
    "Date": df.iloc[split_index]["Date"].values,  
    "Actual Price": y_test.values,  
    "Predicted Price": y_pred
```

```
}}
```

```
print("\nActual vs Predicted Prices:")
```

```
print(comparison.head(10).to_string(index=False))
```

```
# Plot actual vs predicted
```

```
plt.figure(figsize=(12, 6))
```

```
plt.plot(
```

```
    comparison["Date"],
```

```
    comparison["Actual Price"],
```

```
    label="Actual Price"
```

```
)
```

```
plt.plot(
```

```
    comparison["Date"],
```

```
    comparison["Predicted Price"],
```

```
    label="Predicted Price"
```

```
)
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Stock Price")
```

```
plt.title("Actual vs Predicted Stock Prices")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Plot complete stock price history
```

```
plt.figure(figsize=(12, 6))
```

```
plt.plot(  
    df["Date"],  
    df["Close"]  
)
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Closing Price")
```

```
plt.title("Historical Stock Price")
```

```
plt.grid(True)
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("\nExperiment 9 completed successfully.")
```

Original Dataset:

	Date	Open	High	Low	Close	Volume
0	2020-01-01	101.919606	102.838312	98.702620	100.993428	420701
1	2020-01-02	102.421124	103.182544	99.831497	100.511707	379687
2	2020-01-03	101.463339	104.952982	100.785197	102.861907	530634
3	2020-01-04	105.934125	108.200315	104.840885	105.371156	849971
4	2020-01-05	101.935514	103.165122	99.044261	102.586157	596507

Dataset Shape:

(500, 6)

Missing Values:

Date 0
Open 0
High 0
Low 0
Close 0
Volume 0
dtype: int64

Preprocessed Dataset:

	Date	Open	High	Low	Close	Volume	Days	\
1	2020-01-02	102.421124	103.182544	99.831497	100.511707	379687	1	
2	2020-01-03	101.463339	104.952982	100.785197	102.861907	530634	2	
3	2020-01-04	105.934125	108.200315	104.840885	105.371156	849971	3	
4	2020-01-05	101.935514	103.165122	99.044261	102.586157	596507	4	
5	2020-01-06	102.790221	106.117877	100.446450	103.277347	982470	5	

	Previous_Close
1	100.993428
2	100.511707
3	102.861907
4	105.371156
5	102.586157

Training Samples: 399

Testing Samples: 100

Linear Regression model trained successfully.

=====

MODEL EVALUATION

=====

Mean Squared Error (MSE): 0.4713

Root Mean Squared Error (RMSE): 0.6865

R² Score: 0.9817

Actual vs Predicted Prices:

	Date	Actual Price	Predicted Price
	2021-02-04	161.442212	161.438403
	2021-02-05	164.213390	162.992299
	2021-02-06	166.185909	166.418123
	2021-02-07	167.005327	167.569901
	2021-02-08	166.710728	166.471843
	2021-02-09	169.511063	169.691102

2021-02-10	166.731813	165.959139
2021-02-11	169.124183	169.837999
2021-02-12	170.125577	169.776619
2021-02-13	171.319065	171.001250



CODE 2:

EXPERIMENT 9

Time Series Forecasting using Linear Regression

Dataset: Air Passengers

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

Create Air Passengers dataset

passengers = [

112, 118, 132, 129, 121, 135, 148, 148, 136, 119, 104, 118,
115, 126, 141, 135, 125, 149, 170, 170, 158, 133, 114, 140,
145, 150, 178, 163, 172, 178, 199, 199, 184, 162, 146, 166,
171, 180, 193, 181, 183, 218, 230, 242, 209, 191, 172, 194,
196, 196, 236, 235, 229, 243, 264, 272, 237, 211, 180, 201,
204, 188, 235, 227, 234, 264, 302, 293, 259, 229, 203, 229,
242, 233, 267, 269, 270, 315, 364, 347, 312, 274, 237, 278,
284, 277, 317, 313, 318, 374, 413, 405, 355, 306, 271, 306,
315, 301, 356, 348, 355, 422, 465, 467, 404, 347, 305, 336,
340, 318, 362, 348, 363, 435, 491, 505, 404, 359, 310, 337,
360, 342, 406, 396, 420, 472, 548, 559, 463, 407, 362, 405,
417, 391, 419, 461, 472, 535, 622, 606, 508, 461, 390, 432,
441, 419, 461, 472, 535, 622, 606, 508, 461, 390, 432, 441,

```
417, 391, 419, 461, 472, 535, 622, 606, 508, 461, 390, 432  
]
```

```
# Use first 144 months
```

```
passengers = passengers[:144]
```

```
# Create monthly dates
```

```
dates = pd.date_range(  
    start="1949-01-01",  
    periods=len(passengers),  
    freq="MS"  
)
```

```
# Create DataFrame
```

```
df = pd.DataFrame({  
    "Date": dates,  
    "Passengers": passengers  
})
```

```
print("Air Passengers Dataset:")
```

```
print(df.head())
```

```
print("\nDataset Shape:")
```

```
print(df.shape)
```

```
# Check missing values
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
# Convert Date to datetime
df["Date"] = pd.to_datetime(df["Date"])

# Sort chronologically
df = df.sort_values("Date")

# Create time feature
df["Month_Number"] = np.arange(len(df))

# Create year and month features
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month

print("\nPreprocessed Dataset:")
print(df.head())

# Plot historical passenger data
plt.figure(figsize=(12, 6))

plt.plot(
    df["Date"],
    df["Passengers"]
)

plt.xlabel("Date")
plt.ylabel("Number of Passengers")
plt.title("Monthly Air Passenger Data")

plt.grid(True)
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Select features
```

```
X = df[["Month_Number"]]
```

```
y = df["Passengers"]
```

```
# Chronological 80-20 split
```

```
split_index = int(len(df) * 0.8)
```

```
X_train = X.iloc[:split_index]
```

```
X_test = X.iloc[split_index:]
```

```
y_train = y.iloc[:split_index]
```

```
y_test = y.iloc[split_index:]
```

```
print("\nTraining Samples:", len(X_train))
```

```
print("Testing Samples:", len(X_test))
```

```
# Create Linear Regression model
```

```
model = LinearRegression()
```

```
# Train model
```

```
model.fit(
```

```
    X_train,
```

```
        y_train
    )

    print("\nLinear Regression model trained successfully.")

    # Predict test data
    y_pred = model.predict(X_test)

    # Calculate MSE
    mse = mean_squared_error(
        y_test,
        y_pred
    )

    # Calculate RMSE
    rmse = np.sqrt(mse)

    # Calculate R2
    r2 = r2_score(
        y_test,
        y_pred
    )

    print("\n" + "=" * 50)
    print("MODEL EVALUATION")
    print("=" * 50)

    print("Mean Squared Error (MSE):")
    print(round(mse, 4))
```

```
print("\nRoot Mean Squared Error (RMSE):")
```

```
print(round(rmse, 4))
```

```
print("\nR2 Score:")
```

```
print(round(r2, 4))
```

```
# Create comparison dataframe
```

```
comparison = pd.DataFrame({  
    "Date": df.iloc[split_index]["Date"].values,  
    "Actual Passengers": y_test.values,  
    "Predicted Passengers": y_pred  
})
```

```
print("\nActual vs Predicted Values:")
```

```
print(comparison.head(15).to_string(index=False))
```

```
# Plot actual vs predicted
```

```
plt.figure(figsize=(12, 6))
```

```
plt.plot(  
    comparison["Date"],  
    comparison["Actual Passengers"],  
    label="Actual"  
)
```

```
plt.plot(  
    comparison["Date"],  
    comparison["Predicted Passengers"],
```

```
        label="Predicted"
    )

plt.xlabel("Date")
plt.ylabel("Number of Passengers")
plt.title("Actual vs Predicted Air Passengers")

plt.legend()
plt.grid(True)

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# Predict future 12 months
future_months = np.arange(
    len(df),
    len(df) + 12
).reshape(-1, 1)

future_predictions = model.predict(
    future_months
)

future_dates = pd.date_range(
    start=df["Date"].max() + pd.DateOffset(months=1),
    periods=12,
```

```

    freq="MS"
)

future_df = pd.DataFrame({
    "Date": future_dates,
    "Predicted Passengers": future_predictions
})

print("\nFuture 12-Month Predictions:")
print(future_df.to_string(index=False))

print("\nExperiment 9 completed successfully.")

```

OUTPUT 2:

Air Passengers Dataset:

	Date	Passengers
0	1949-01-01	112
1	1949-02-01	118
2	1949-03-01	132
3	1949-04-01	129
4	1949-05-01	121

Dataset Shape:

(144, 2)

Missing Values:

Date	0
Passengers	0

dtype: int64

Preprocessed Dataset:

	Date	Passengers	Month_Number	Year	Month
0	1949-01-01	112	0	1949	1
1	1949-02-01	118	1	1949	2
2	1949-03-01	132	2	1949	3
3	1949-04-01	129	3	1949	4
4	1949-05-01	121	4	1949	5

Training Samples: 115

Testing Samples: 29

Linear Regression model trained successfully.

=====

MODEL EVALUATION

=====

Mean Squared Error (MSE):
5447.1636

Root Mean Squared Error (RMSE):
73.8049

R² Score:
0.1077

Actual vs Predicted Values:

Date	Actual Passengers	Predicted Passengers
1958-08-01	505	385.735011
1958-09-01	404	388.248584
1958-10-01	359	390.762156
1958-11-01	310	393.275728
1958-12-01	337	395.789300
1959-01-01	360	398.302872
1959-02-01	342	400.816444
1959-03-01	406	403.330017
1959-04-01	396	405.843589
1959-05-01	420	408.357161
1959-06-01	472	410.870733
1959-07-01	548	413.384305
1959-08-01	559	415.897877
1959-09-01	463	418.411450
1959-10-01	407	420.925022

Future 12-Month Predictions:

Date	Predicted Passengers
1961-01-01	458.628604
1961-02-01	461.142176
1961-03-01	463.655748
1961-04-01	466.169321
1961-05-01	468.682893
1961-06-01	471.196465
1961-07-01	473.710037
1961-08-01	476.223609
1961-09-01	478.737181
1961-10-01	481.250754
1961-11-01	483.764326
1961-12-01	486.277898

